# Rapport: Scrabble en Java

Alexandre Lou-Poueyou
Aymeric Cassard
Ludovic Andrieu
Jean Mayol
Mathéo Petry
Valentin Cazet

29 janvier 2025

## Table des matières

**1 Le contexte et l'existant du projet** **2**
1.1 Choix du jeu et du langage . . . . . 2
1.1.1 Pourquoi le scrabble ? . . . . . 2
1.1.2 Java . . . . . 2
1.2 Existant du projet . . . . . 2

**2 Une explication détaillée du sujet** **4**
2.1 Règles du jeu . . . . . 4
2.2 Structures de données . . . . . 5
2.3 Algorithmes spécifiques . . . . . 6

**3 La liste des besoins visés** **7**

**4 Un agenda prévisionnel des livraisons des besoins** **9**

**5 Architecture du projet** **12**
5.1 Modules du projet . . . . . 12
5.2 Dépendances du projet . . . . . 13

**6 Les spécifications étendues** **13**

# 1 Le contexte et l'existant du projet

## 1.1 Choix du jeu et du langage

### 1.1.1 Pourquoi le scrabble ?

Notre choix pour le projet s'est porté sur le Scrabble. Cela nous a semblé être le meilleur choix pour notre groupe car ce jeu est très répandu et que tout le monde connaît les règles ; de plus, celles-ci ne possèdent pas de cas particuliers complexes à d'autres jeux comme les échecs ou le Tafl ce qui nous permet de nous concentrer sur des aspects plus techniques de la gestion et création du projet (Génie logiciel, IA, Interface graphique, bon déroulement du jeu).

### 1.1.2 Java

Le langage de programmation ayant retenu notre attention est Java grâce à sa richesse, son accessibilité et sa lisibilité, de par sa documentation très complète et ses nombreuses librairies robustes qui peuvent nous simplifier diverses tâches. Enfin, la matière "Approche Objet" que nous avons suivie le semestre suivant est un plus pour la bonne conduite de ce projet notamment via les techniques de Design Pattern découvertes durant le semestre.

De plus, nous avons comparé Java avec les autres langages disponibles (Python, C) et nous avons repéré des points nous mettant d'autant plus dans l'idée d'utiliser Java :

Python :

— Langage plus lent

— Importations des fichiers plus sujets à problème

— Typage spécifique de Python pouvant être plus compliqué qu'autre chose vis à vis de la taille du projet

C :

— Aucune structure de données implémentées de base

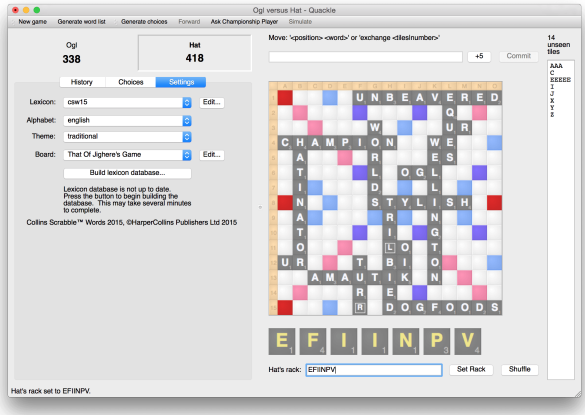
— Gestion de la mémoire

— Pas de possibilité de faire de la programmation orienté objet

## 1.2 Existant du projet

Le scrabble étant un jeu très connu et répandu, il n'est pas étonnant de trouver des existants de ce jeu, principalement pour ce qui est du test de performances d'algorithmes. On peut retrouver dans cette situation le "Quackle" [3] développé

par le MIT et étant une implémentation du scrabble avec une IA utilisant l'algorithme gaddag pour lui permettre de jouer le meilleur coup possible.




Mais on peut aussi retrouver des existants du projet n'étant pas orientés que vers la recherche, on peut aussi retrouver des implémentations du Scrabble se concentrant sur l'aspect ludique et distrayant via l'ajout de différents modes de jeu, d'un aspect compétitif ou d'éléments de customisation comme les jeux Scrabble développés pour les mobiles (exemple : Scrabble Go pour les téléphones sous IOS).



# 2 Une explication détaillée du sujet

## 2.1 Règles du jeu

Le Scrabble est un jeu de société qui consiste à former des mots sur une grille carrée de 15x15 cases. Les 100 lettres utilisées ont une valeur de point choisie selon la fréquence du mot dans la langue. 2 lettres 'Joker' supplémentaires sont présentes, elles permettent au joueur de placer n'importe quelle lettre, mais n'ajoutent aucun point au mot. Chaque joueur dispose de lettres tirées aléatoirement qu'il place sur la grille pour marquer des points.
La grille contient des cases bonus :

— Lettre double : double la valeur de la lettre posée.

— Lettre triple : triple la valeur de la lettre posée.

— Mot double : double la valeur du mot posé.

— Mot triple : triple la valeur du mot posé.

Les multiplicateurs de mots se combinent dans le cas où le mot posé touche plusieurs cases.

Le mot doit exister dans le dictionnaire courant du programme pour pouvoir être posé. Les éventuels mots adjacents au mot posé rapportent des points.

Chaque joueur dispose de 7 lettres dans sa main. Si un joueur pose toutes ses lettres en un tour, alors s'applique un bonus de 50 points.
Les actions que peut entreprendre le joueur lors de son tour sont :

— Poser un mot.

— Échanger des lettres de la pioche.

— Passer son tour.

Les conditions de fin de partie sont les suivantes :

— **Reliquat épuisé** : Un joueur a épuisé toutes ses lettres et la pioche est vide.

— **Blocage** : Les joueurs ne trouvent plus de mots a jouer et il reste moins de 7 lettres dans la pioche. Les valeurs des lettres de leur mains sont déduites de leur score final.

— **Temps épuisé** : Les deux joueurs n'ont plus de temps (variante Blitz uniquement).

Le joueur avec le score le plus élevé est le gagnant.

*Variantes*

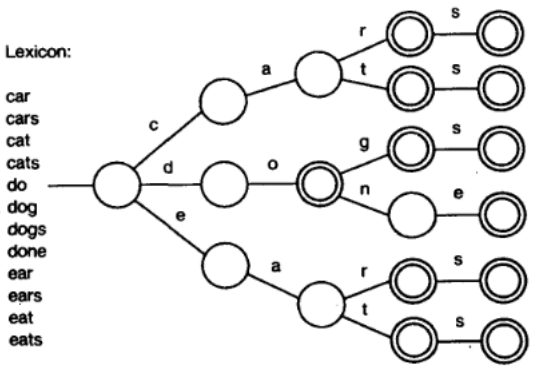
— **Super-scrabble** : La grille fait 21x21.

— **Blitz** : Chaque joueur dispose d'un temps imparti, qui s'écoule lors de son tour. Si un joueur n'a plus de temps, il ne peut plus jouer.

## 2.2 Structures de données

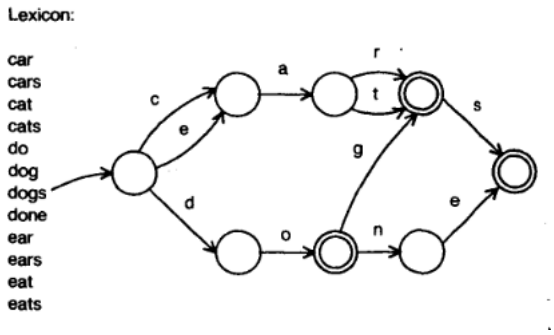
*Structures de données simples*
L'utilisation de Java dans le projet permet d'éviter d'implémenter par nous-mêmes les structures de données de base du projet. Par exemple, la pioche est représentée par une HashTable qui associe chaque lettre à un compteur, représentant le nombre de lettres restantes dans la pioche.

*Représentation du dictionnaire*
Une manière commune de représentation de lexique est un arbre pour lequel chaque arête est une lettre du mot, et certains nœuds sont marqués comme finaux, indiquant que le chemin jusqu'à ce nœud constitue un mot. Cette structure est communément appelée un trie.



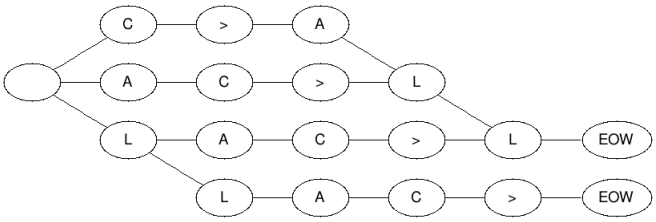

Le trie n'est pas utilisé dans notre projet mais aide à visualiser le fonctionnement des structures de données suivantes.

Le **DAWG**(Directed Acyclic Word-Graph) [3] nous permet de représenter le dictionnaire de validité des mots en mémoire. Il s'agit d'un automate fini pour lequel le langage est l'ensemble des mots du dictionnaire, et les transitions sont les lettres que l'on ajoute pour former le mot. Enfin, les états finaux sont les différents mots du lexique.




Cette structure de données est équivalente au trie en termes de fonctionnalités, mais prend moins de place en mémoire.

Le **GADDAG** [2] est une autre manière de représenter le lexique. Dans cette structure, chaque mot est représenté à partir de chacune de ses lettres intermédiaires.




Cette structure est une optimisation du DAWG en terme de calcul des mots possibles tout en respectant les contraintes du scrabble. Le GADDAG est donc utilisé dans les algorithmes de recherche de meilleur coup par IA. Cependant, cette structure prend plus de place en mémoire que le DAWG.

## 2.3 Algorithmes spécifiques

Nous implémentons deux algorithmes de recherche dans la partie intelligence artificielle du projet. Le Scrabble est un jeu à somme nulle, étant donné qu'il y a un nombre fini de lettres et d'emplacements sur le plateau, donc le coup d'un joueur est une perte nette pour les autres. C'est un prérequis de nombreux algorithmes de recherche du meilleur coup, dont les deux algorithmes suivants :

— **MiniMax** Avec cet algorithme, on explore un arbre de décision où chaque niveau alterne entre les coups du joueur (maximisant le score) et ceux de l'adversaire (minimisant le score). Les nœuds finaux de l'arbre correspondent à une évaluation heuristique de l'état du jeu dans le cas où l'algorithme n'a

pu aller assez en profondeur pour calculer une véritable fin de partie. On joue ensuite le coup menant au meilleur coup trouvé lors de la recherche.

— **Monte-Carlo** Dans l'algorithme de Monte-Carlo [1], on parcourt toujours un arbre de décision, cependant on va parcourir des chemins de manière aléatoire, au lieu d'une recherche en profondeur classique, jusqu'à tomber sur une véritable fin de partie. Le point fort de cet algorithme est qu'il ne nécessite pas de fonction d'évaluation du plateau pour être efficace, ce qui est avantageux dans le cadre du Scrabble, étant donné qu'il s'agit d'un jeu à information incomplète jusqu'aux derniers tours, ce qui complique la mise en place d'une fonction d'évaluation [5].

# 3 La liste des besoins visés

| | Besoin | Dépendance |
|---|---|---|
| B01 | Mettre en place une AI | B07, B13, B10 |
| B02 | Lancer le programme grâce à la commande scrabble | B12 |
| B03 | Pouvoir jouer en plusieurs langues | B07 |
| B04 | Avoir une interface graphique GUI | B02, B13 |
| B05 | Avoir une interface de jeux en ligne de commande CLI | B02, B13 |
| B06 | Pouvoir accéder à un historique des coups joués commentés | B13 |
| B07 | Déterminer si un mot est dans le dictionnaire | |
| B08 | L'application ne doit jamais crasher de façon non gérée | B11 |
| B09 | Pouvoir jouer en multijoueur local | B04, B13 |
| B10 | Avoir un système de score | B13 |
| B11 | Mettre en place des tests et les automatisés | |
| B12 | Respecter un fichier de configuration au lancement de l'appli | |
| B13 | Gérer le déroulement d'une partie | B07, B08 |
| B14 | Pouvoir sauvegarder et charger une partie | B13 |
| B15 | Support de différents modes de jeux | B13 |

— **B01 : Mettre en place une AI**
Il faut que le joueur puisse jouer contre l'IA et que l'IA puissent trouver les meilleurs coups.

— **B02 : Lancer le programme grâce à la commande scrabble**
Il faut pouvoir lancer exécuter le programme a l'aide de cette commande

— **B03 : Pouvoir jouer en plusieurs langues**
Ce qui signifie d'avoir accès aux dictionnaire correspondant à la langue

— **B04 : Avoir une interface graphique GUI**
Afin que l'utilisateur est un visuel du jeux

— **B05 : Avoir une interface de jeux en ligne de commande CLI**
Pour pouvoir jouer via la console

— **B06 : Pouvoir accéder à un historique des coups joués commentés**
Pour se souvenir qui a poser tel mot ainsi que leur score

— **B07 : Déterminer si un mot est dans le dictionnaire**
Grace au DWAG pour vérifier quels mots sont jouables

— **B08 : L'application ne doit jamais crasher de façon non gérée**
Pour avoir une utilisation fluide de l'application

— **B09 : Pouvoir jouer en multijoueur local**
Car le scrabble est un jeu qui se joue a plusieurs

— **B10 : Avoir un système de score**
Qui détermine qui a gagner. Utile aussi pour l'IA

— **B11 : Mettre en place des tests et les automatisés**
Afin de vérifier si les fonctions créer sont correctes

— **B12 : Respecter un fichier de configuration au lancement de l'appli**
Pouvoir ouvrir un fichier de jeu avec ses paramètres

— **B13 : Gérer le déroulement d'une partie**
Implémentation du jeu (lettres, scores,a qui le tour, ...)

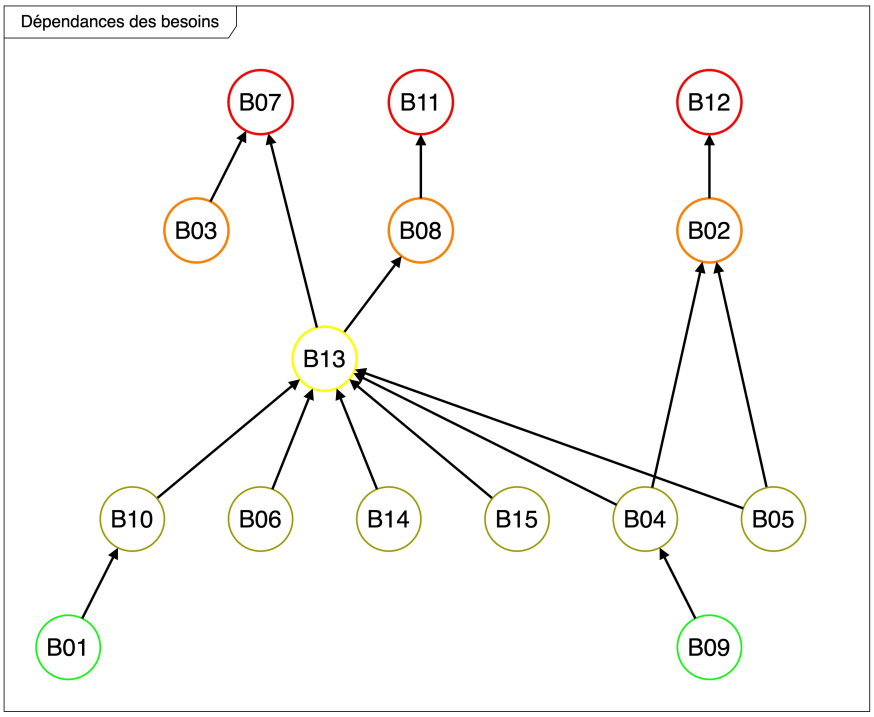
— **B14 : Pouvoir sauvegarder et charger une partie**
afin de pouvoir s'arrêter et reprendre une partie en cours

— **B15 : Support de différents modes de jeux**
Mode Blitz, Mode super-scrabble (plus grand plateaux, plus de lettres)

Ensuite nous avons créé les dépendances de ces besoins. C'est-à-dire de quels besoins dépendent chaque besoin. Cela permet de déterminer ainsi un ordre sur les besoins traités.

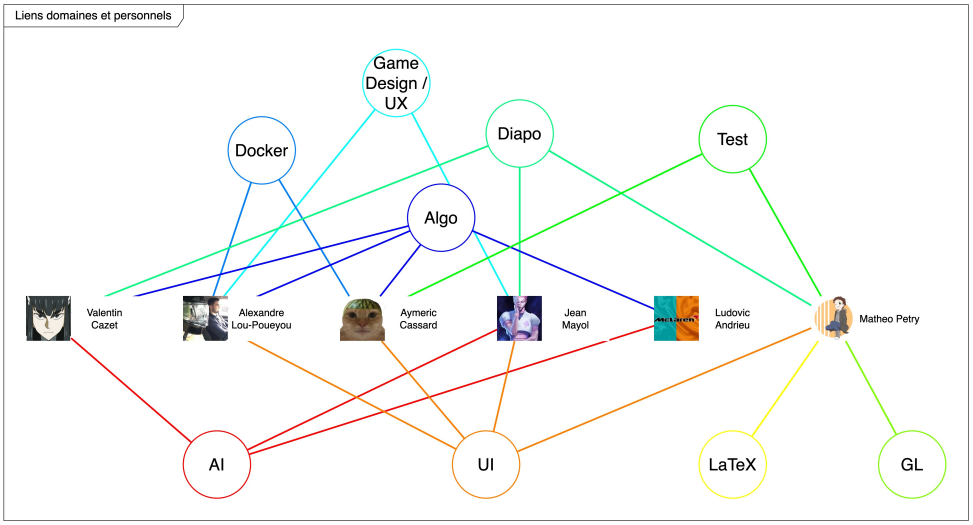
# 4 Un agenda prévisionnel des livraisons des besoins

Pour établir l'agenda prévisionnel, nous allons nous appuyer sur le graphe des dépendances entre les besoins. Ce graphe nous permet d'identifier que certains besoins nécessitent d'autres besoins déjà implémentés. Par exemple, le besoin B01 dépend du besoin B10.

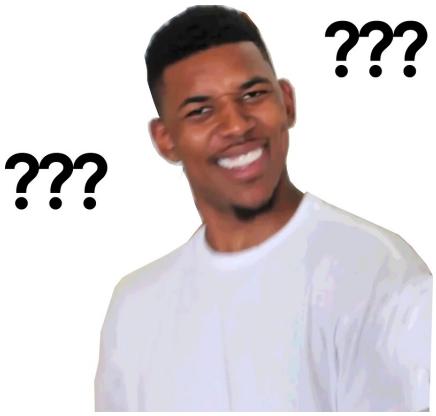



Nous en concluons donc que nous devons d'abord prioriser l'implémentation des besoins B07, B11 et B12.

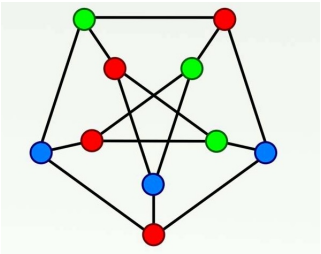
Ensuite, nous avons réalisé un sondage auprès des membres de l'équipe afin de déterminer leurs préférences en matière de développement. Certains souhaitent se concentrer sur l'implémentation de l'AI et des algorithmes. Il est également important de noter que certains ne souhaitent pas s'occuper des phases de test, car ils ne sont pas dans le parcours GL. Il est donc préférable de confier cette tâche aux personnes spécialisées dans ce domaine.




Pour établir notre agenda prévisionnel, nous devons donc combiner le graphe des dépendances avec les préférences de développement des membres de l'équipe. Mais comment faire ?



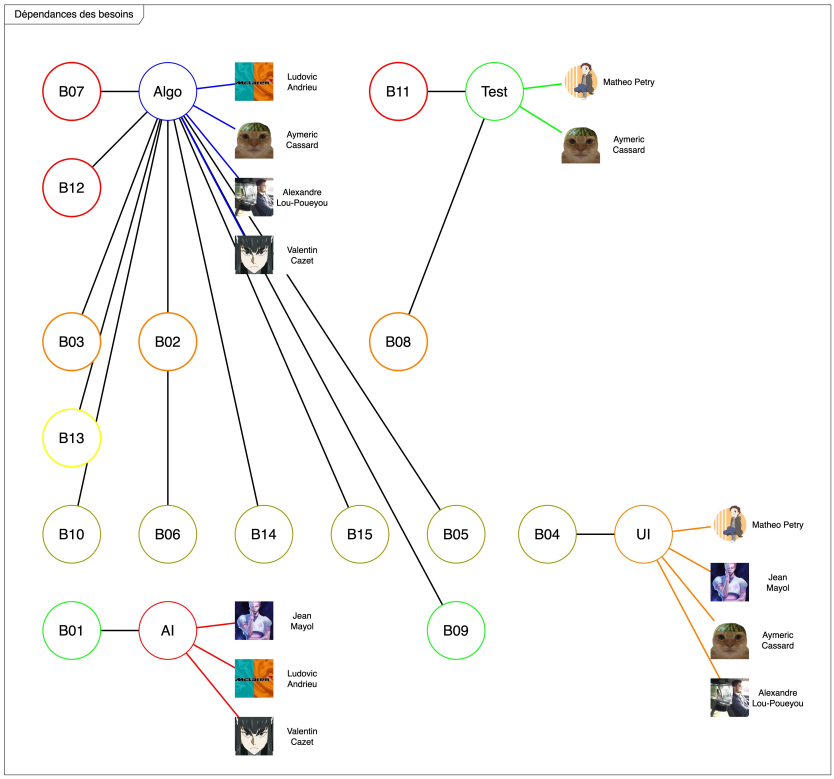

Nous pourrions réduire ce problème à un problème de coloration de graphe. Cependant, nous avons décidé de ne pas suivre cette approche, car cela nécessiterait trop de temps pour le codage, d'implémentation et les tests. Nous avons estimé que ce développement prendrait au moins une heure, alors que le faire manuellement ne prend que 5 minutes.



Nous avons donc choisi de combiner ces deux graphes manuellement, en faisant en sorte qu'un membre de l'équipe prenne en charge les dépendances et les associe ensemble.

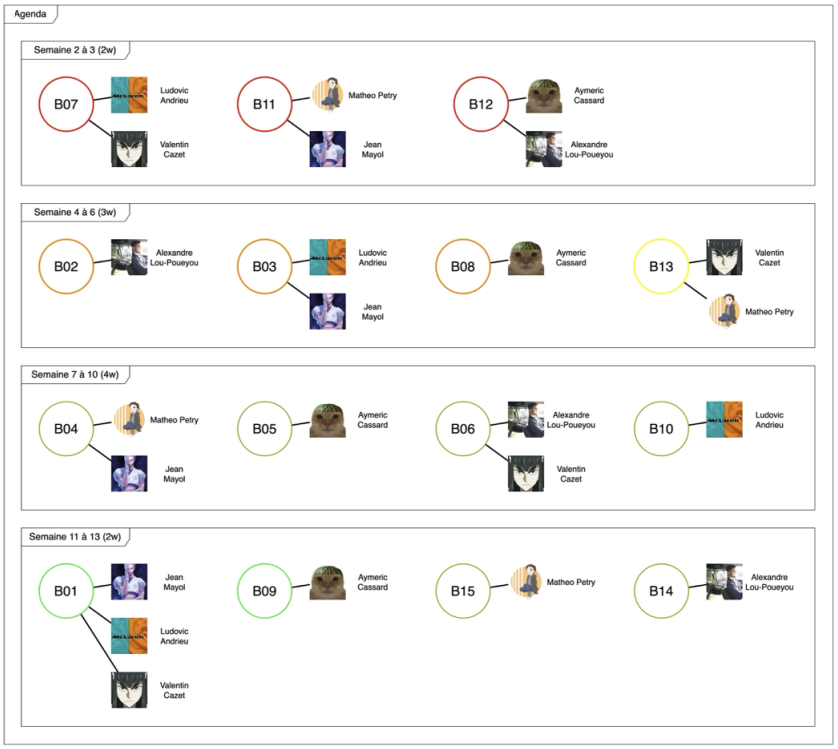


Nous obtenons ainsi un graphe qui combine les besoins, leur ordre de développement, les domaines d'intervention dans un projet, ainsi que les membres de l'équipe.



Il ne reste plus qu'une étape pour générer l'agenda prévisionnel : pour chaque besoin, il faut choisir une personne responsable de son développement.

Nous avons donc établi notre agenda prévisionnel.




Par exemple, de la Semaine 2 à la Semaine 3, soit 2 semaines de travail, le besoin B07 sera développé par Ludovic et Valentin. Il en va de même pour le besoin B11, qui sera pris en charge par Mathéo et Jean, etc.

# 5 Architecture du projet

## 5.1 Modules du projet

Nous avons choisi une architecture de style MVC pour organiser les modules du projet. Ce motif d'architecture a été choisi car il pose des lignes directrices simples et comprises par tous les membres vis-à-vis de la question du placement des différents modules. De plus, il se prête bien à la mise en place de la partie graphique. Conformément à ce modèle, nos modules sont donc découpés en trois parties.

*Modèle*

Le modèle contient les différentes structures de données, et autres éléments logiques relatifs aux contraintes du jeu. Les éléments de cette couche sont : Game, AI.
*Vue*
La vue est constituée de tous les éléments visibles par l'utilisateur. Ici, elle est découpée en deux grands axes. La partie CLI, et la partie GUI.
*Controlleur*
Cette couche de l'architecture traite les interactions avec l'utilisateur. C'est donc cette partie qui s'occupe d'actualiser le modèle selon les différentes actions de l'utilisateur.

## 5.2 Dépendances du projet

Le projet est codé en Java version 17.0.13, et est compilé à l'aide du système de build Maven. Afin de respecter les spécifications du projet, un certain nombre de dépendances sont ajoutées :

- **JUnit** : Framework de test, utilisé pour les différents scénarios de test du code.
- **Jacoco** : Librairie de couverture de code, utilisée en conjonction avec JUnit pour certifier le pourcentage de code source testé.
- **Commons-cli** : API d'analyse d'options de commandes. Utilisé pour interpreter les options passées en ligne de commande.
- **JavaFx** : Framework d'interface utilisateur, utilisé pour la partie interface graphique.
- **Checkstyle** : Outil d'analyse de code permettant de vérifier la conformité du formatage avec le style de codage de Google.

# 6 Les spécifications étendues

Les besoins sont composés d'exigences. Les exigences sont des règles plus précises et plus strictes. Ces exigences sont elles-mêmes composées de tâches qui doivent être traitées. Cela permet d'avoir une visualisation plus précise sur ce qu'il faut faire et des tâches à effectuer d'abord. Voir ci-dessous la liste des premières exigences et tâches à être traitées. Elles correspondent aux besoins devant être traités en premier, comme vu précédemment. On peut s'apercevoir que l'exigence E07-05 manque ou que l'exigence E11-01 n'a pas de tâche : cela est dû au fait que nous avons dû faire des modifications pendant le projet et donc nous adapter (par exemple, nous avons supprimé une exigence car elle n'était pas nécessaire). C'est ainsi que nous avons pour l'instant seulement les besoins B07, B11 et B12.

Ce sont les premiers à devoir être traités. Nous créerons au fur et à mesure les exigences et les tâches pour les futurs besoins.

- B07 : Déterminer si un mot est dans le dictionnaire
  - E07-01 : charger une liste de mots à partir d'un fichier
    - ☐ T07-01-01 : charger le texte depuis un fichier (attention à l'encodage)
    - ☐ T07-01-02 : séparer le texte en Mots
  - E07-02 : construire le dictionnaire à partir d'une liste de mots (déjà simplifiés)
    - ☐ T07-02-01 : créer un dictionnaire vide
    - ☐ T07-02-02 : ajouter un mot au dictionnaire
    - ☐ T07-02-03 : définir une interface pour les dictionnaires
  - E07-03 : tester si un mot est dans le dictionnaire
    - ☐ T07-03-01 : implémenter l'interface Collection pour les dictionnaires
  - E07-04 : définir une classe de Mots
    - ☐ T07-04-01 : construire un mot à partir d'un String, simplifier les lettres spéciales (alphabets latins seulement)
  - E07-06 : implémenter un dictionnaire DAWG
    - ☐ T07-06-01 : implémenter une nouvelle classe dictionnaire qui implémente ajout d'un mot avec DAWG
- B12 : Respecter un fichier de configuration au lancement de l'appli
  - E12-01 : définir un format pour la configuration
    - ☐ T12-01-01 : Écrire la classe configuration avec ses états par défauts (default, blitz, super scrabble)
  - E12-02 : charger le fichier de configuration
    - ☐ T12-02-01 : être capable d'extraire une configuration (object) depuis un fichier [4]
  - E12-03 : faire en sorte que les paramètres de la game s'adaptent au fichier de configuration
    - ☐ T12-03-01 : Créer un fichier de configuration capable d'être passé au jeu
    - ☐ T12-03-02 : pouvoir combiner des configurations
  - E12-04 : pouvoir sauvegarder une configuration sans un fichier

        - ☐ T12-04-01 : écrire une méthode qui sauvegarde une configuration dans un fichier

- B11 : Mettre en place des tests et les automatiser
    - E11-01 : définir un ensemble de test
    - E11-02 : gérer les erreurs génériques
        - ☐ T11-02-01 : mettre en place une énumération pour lister les différentes types d'erreurs
        - ☐ T11-02-02 : mettre en place un classe unique pour gérer les erreurs dans notre code
        - ☐ T11-02-03 : implémenter des méthodes pour récuperer l'erreur sous forme de texte

# Références


[1] C. B. Browne, E. Powley, D. Whitehouse, S. M. Lucas, P. I. Cowling, P. Rohlfshagen, S. Tavener, D. Perez, S. Samothrakis, and S. Colton. A survey of monte carlo tree search methods. *IEEE Transactions on Computational Intelligence and AI in games*, 4(1) :1–43, 2012.

[2] S. A. Gordon. A faster move generation algorithm. *SOFTWARE—PRACTICE AND EXPERIENCE*, 24(2) :219–232, February 1994.

[3] A. W. A. . G. J. Jacobson. The world's fastest scrabble program. *Programming Techniques and Data Structures*, 31 :572–578, May 1988.

[4] S. J. Metsker. *Building Parsers with Java*. Addison-Wesley Professional, 2001.

[5] B. Sheppard. World-championship-caliber scrabble. *Artificial Intelligent*, 134, January 2002.